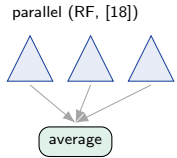
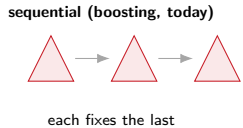
From averaging to fixing mistakes

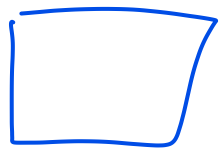
The random forest ([18]) grew trees **in parallel** and averaged them to cut variance. **What if each new tree instead fixes what the previous ones got wrong?**





That sequential "fix the leftover error" idea is **boosting** – the method that ruled tabular ML for a decade.

Outline



▶ սկզբից ես, հետո դու, հետո ես

▶ watch on YouTube

The boosting idea

Combine many weak learners, each fixing what the others miss.

The theoretical question: weak ⇒ strong?

Can a set of **weak** *learners – each only slightly better than a coin flip (say 51% accurate) – be combined into one* **arbitrarily strong** *learner?*

(posed by Kearns & Valiant, 1988.)

Schapire (1990): yes – and constructively. "Weak learnability" and "strong learnability" turn out to be **equivalent**: if you can reliably do a hair better than chance, you can be boosted to *any* accuracy. That was a genuine surprise – and **boosting** is the algorithm that does it, each learner focusing on what the previous ones got wrong.

The "weak learner" is a **shallow tree**, often a **stump** – the "atom" from [17]. We add many, one at a time. (Next: AdaBoost, the first algorithm to pull it off.)

Bagging vs boosting

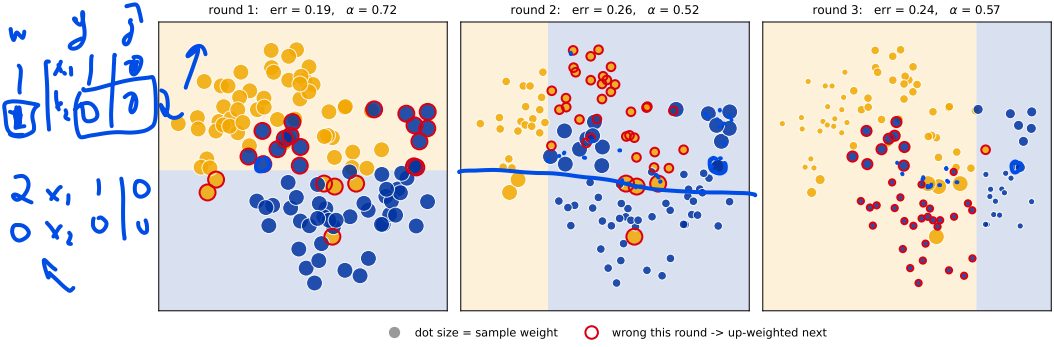
	Bagging / RF ([18])	**Boosting (today)**
trees built	in **parallel**, independent	in **sequence**, dependent
each tree sees	a bootstrap resample	the previous ones' **errors**
combine by	averaging / voting	adding (a weighted sum)
mainly cuts	variance	bias
more trees	safe (plateaus)	**can overfit**

Mirror of [18]. RF bags **deep** (high-variance) trees and averages the variance away. Boosting boosts **shallow** (high-bias) trees – each only nudges the fit, and the bias is removed *across rounds*. Opposite ingredient, opposite target.

AdaBoost: reweight the mistakes

The original boosting algorithm (Freund & Schapire, 1997). Each round:

1. fit a **stump** on the current sample weights;
2. find the points it got **wrong** (red rings below);
3. **up-weight** those, down-weight the rest, and refit.



Each stump is forced to attend to what the previous ones kept missing – the weights carry the "leftover error" from round to round.

AdaBoost, step by step (Freund & Schapire, 1997)

Start with equal weights $w_i = 1/n$. For each round $m = 1, \dots, M$:

1. **Fit** a stump h_m to the *weighted* data.
2. **Weighted error:** $\mathrm{err}_m = \dfrac{\sum_i w_i \mathbb{1}[h_m(\mathbf{x}_i) \neq y_i]}{\sum_i w_i}$ – the weight sitting on the misclassified points.
3. **Vote weight:** $\alpha_m = \frac{1}{2} \ln \dfrac{1 - \mathrm{err}_m}{\mathrm{err}_m}$ – small error $\Rightarrow$ large α_m (a louder vote).
4. **Reweight:** multiply the **misses** by e^{α_m} and the rest by $e^{-\alpha_m}$, then renormalize so the weights sum to 1.

Final: $H(\mathbf{x}) = \mathrm{sign}\Big(\sum_m \alpha_m \, h_m(\mathbf{x})\Big)$.

Two levers: accurate stumps earn a **louder vote** (α_m); misclassified points get **heavier**, so the next stump is forced to attend to them.

AdaBoost by hand: one round

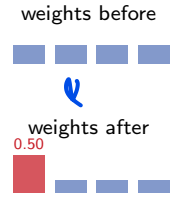
Four points, equal weights $w_i = 0.25$. The round-1 stump gets **one** of them wrong:

$$\text{err}_1 = 0.25, \quad \alpha_1 = \tfrac{1}{2} \ln \tfrac{0.75}{0.25} = \tfrac{1}{2} \ln 3 \approx \mathbf{0.55}.$$

Reweight, then renormalize by the total 0.87:

$$\text{miss: } 0.25\, e^{0.55} = 0.43 \;\to\; \mathbf{0.50},$$

$$\text{each right: } 0.25\, e^{-0.55} = 0.14 \;\to\; \mathbf{0.17}.$$



One mistake now carries **half** the total weight – round 2's stump cannot ignore it. That is the whole trick: each round leans harder on what the last one missed.

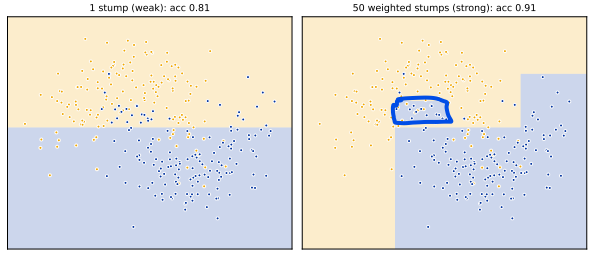
AdaBoost: many weak stumps, one strong vote

The prediction is a **vote weighted by each stump's accuracy**:

$$H(\mathbf{x}) = \operatorname{sign}\Big(\sum_m \alpha_m\, h_m(\mathbf{x})\Big),$$

with $\alpha_m = \frac{1}{2}\ln\frac{1-\mathrm{err}_m}{\mathrm{err}_m}$ – more accurate stumps get a **louder** vote (derived in the HW).

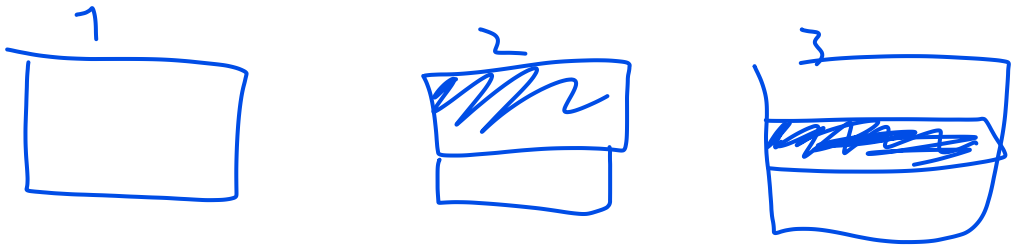
A single stump is weak (one straight cut); **50** of them, weighted, bend into the true shape – the weak-to-strong promise, delivered.



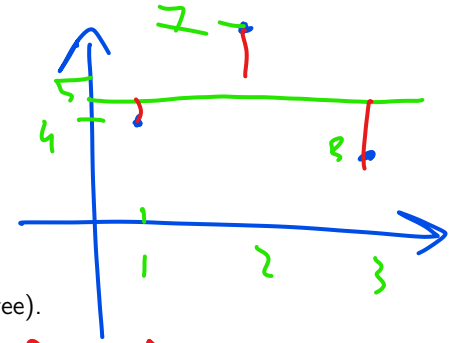
AdaBoost = boosting with the exponential loss. Gradient boosting (next) generalizes it to *any* loss – both are the same "forward stagewise additive model" (AdaBoost just predates the gradient view).

Gradient boosting

Fit the leftover error, add a slice, repeat – and watch it work.



The idea: fit the residuals



Predict apartment rent from area. The recipe:

1. Start simple: $F_0 = \text{mean}(y)$.
2. Compute **residuals** $r = y - F_0$ – what is still wrong.
3. Fit a **small tree** to the residuals.
4. Add it, shrunk by a **learning rate** η: $F_1 = F_0 + \eta\,(\text{tree})$.
5. Recompute residuals; repeat.

$F_0 + \eta \cdot F_1$ $f_0 + f_1 = 5$

Each tree learns the *previous* ensemble's mistakes. The fit creeps toward the data one correction at a time.

$F_0 = \overline{y}$

By hand: round 1 (Yerevan rent vs area)

Init: $F_0 = \text{mean}(y) = \mathbf{304}$ kAMD (a flat line).

Residuals: $r_i = y_i - 304$.

First stump asks "area ≤ 100?" and predicts each side's *residual mean*:

$$\text{left} = -17, \qquad \text{right} = +78.$$

Update with $\eta = 0.4$:

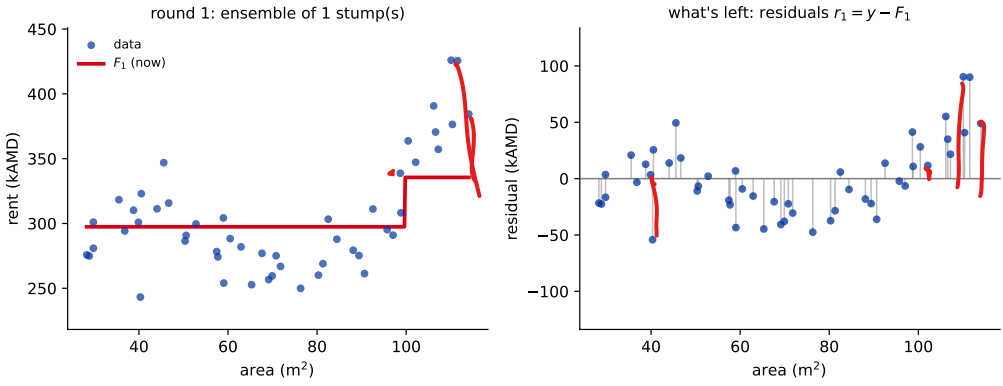
$$F_1 = 304 + 0.4 \times \begin{cases} -17 & \text{area} \leq 100 \\ +78 & \text{area} > 100 \end{cases}$$

$$\Rightarrow \; F_1 = \begin{cases} \mathbf{297} \\ \mathbf{336} \end{cases}$$

One stump nudges the flat line into **two steps**. The big positive residuals at large area shrink first – the model fixes its worst errors first.

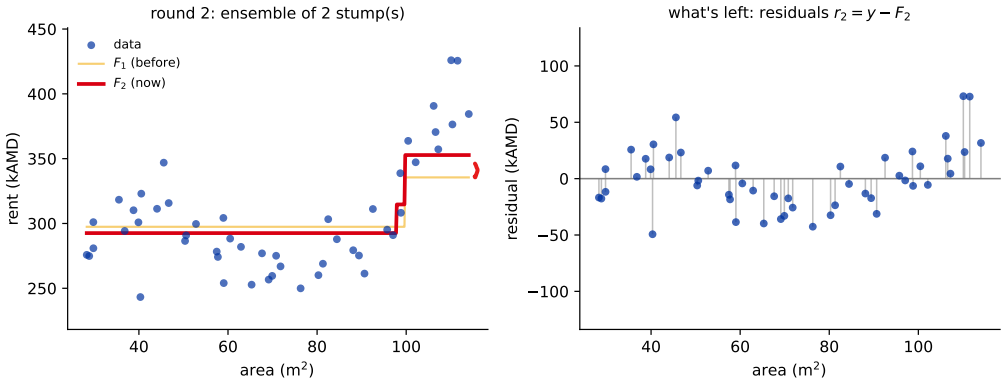
These are the real numbers behind round 1 of the next slide.

Watch it fit: gradient boosting, round by round



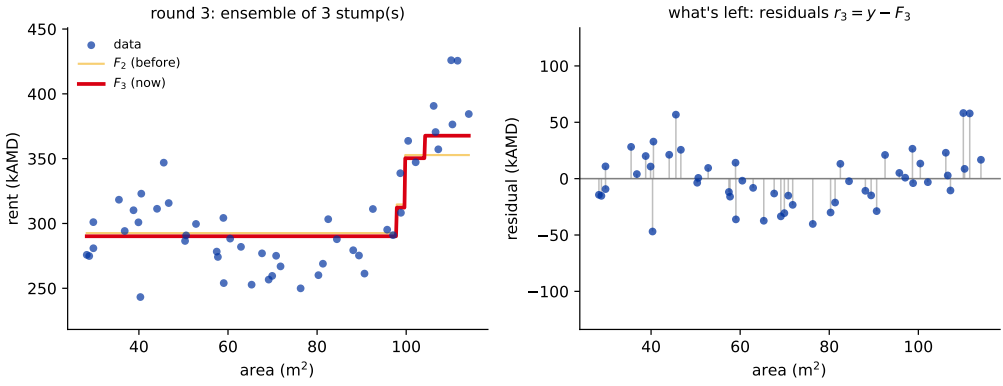
Left: data + the cumulative fit F_k (faint = F_{k-1}, so you see the stump just added). **Right:** the residuals $r_k = y - F_k$ collapsing toward zero. *(Step through rounds 1–6, then jump to round 50: the fit has stopped moving.)*

Watch it fit: gradient boosting, round by round



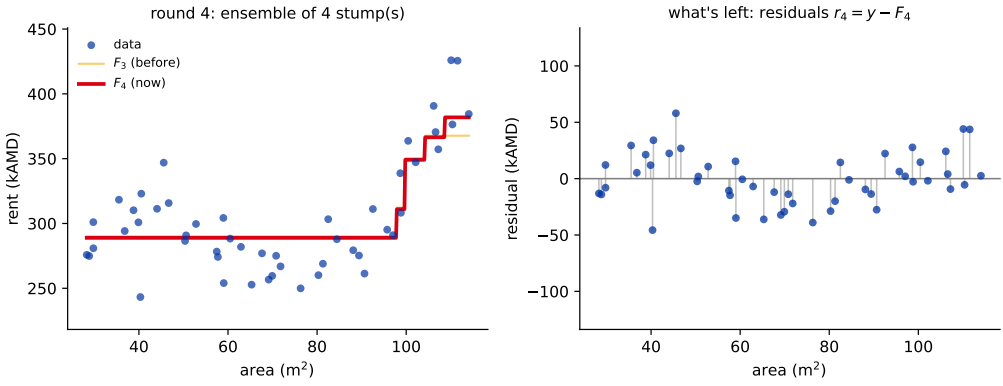
Left: data + the cumulative fit F_k (faint = F_{k-1}, so you see the stump just added). **Right:** the residuals $r_k = y - F_k$ collapsing toward zero. *(Step through rounds 1–6, then jump to round 50: the fit has stopped moving.)*

Watch it fit: gradient boosting, round by round



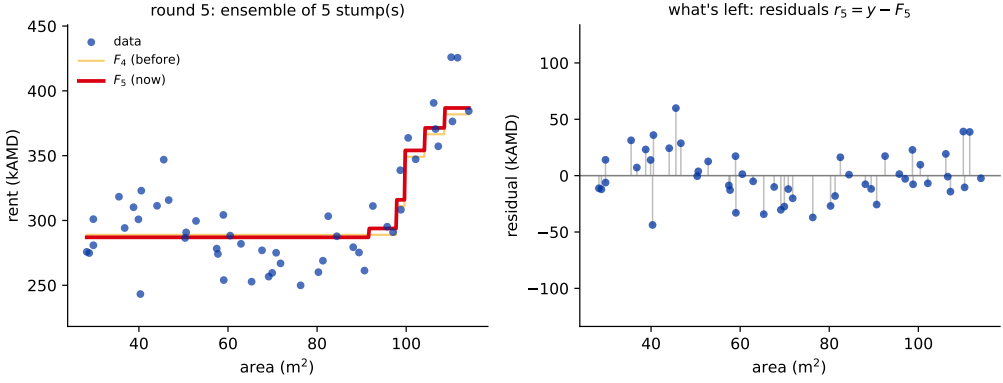
Left: data + the cumulative fit F_k (faint = F_{k-1}, so you see the stump just added). **Right:** the residuals $r_k = y - F_k$ collapsing toward zero. *(Step through rounds 1–6, then jump to round 50: the fit has stopped moving.)*

Watch it fit: gradient boosting, round by round



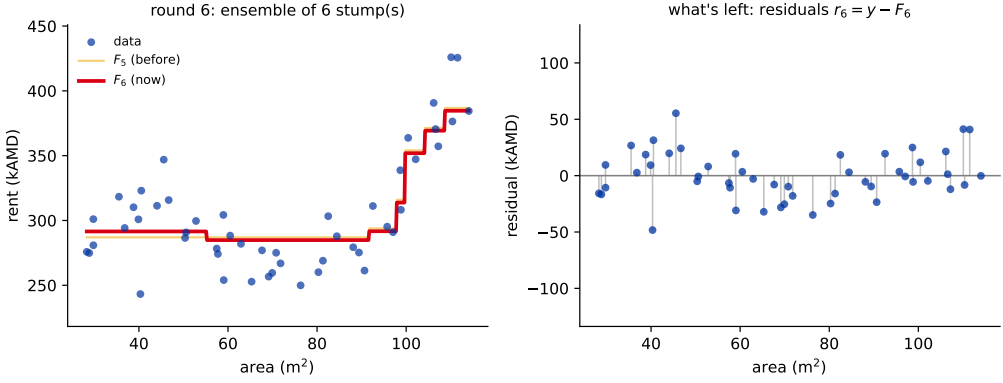
Left: data + the cumulative fit F_k (faint = F_{k-1}, so you see the stump just added). **Right:** the residuals $r_k = y - F_k$ collapsing toward zero. *(Step through rounds 1–6, then jump to round 50: the fit has stopped moving.)*

Watch it fit: gradient boosting, round by round



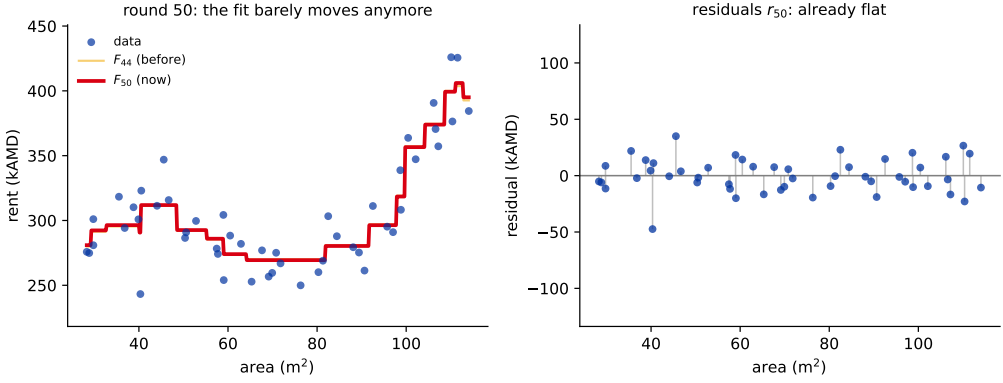
Left: data + the cumulative fit F_k (faint = F_{k-1}, so you see the stump just added). **Right:** the residuals $r_k = y - F_k$ collapsing toward zero. *(Step through rounds 1–6, then jump to round 50: the fit has stopped moving.)*

Watch it fit: gradient boosting, round by round



Left: data + the cumulative fit F_k (faint = F_{k-1}, so you see the stump just added). **Right:** the residuals $r_k = y - F_k$ collapsing toward zero. *(Step through rounds 1–6, then jump to round 50: the fit has stopped moving.)*

Watch it fit: gradient boosting, round by round



Left: data + the cumulative fit F_k (faint = F_{k-1}, so you see the stump just added). **Right:** the residuals $r_k = y - F_k$ collapsing toward zero. *(Step through rounds 1–6, then jump to round 50: the fit has stopped moving.)*

Predict-first: does boosting overfit with more trees?

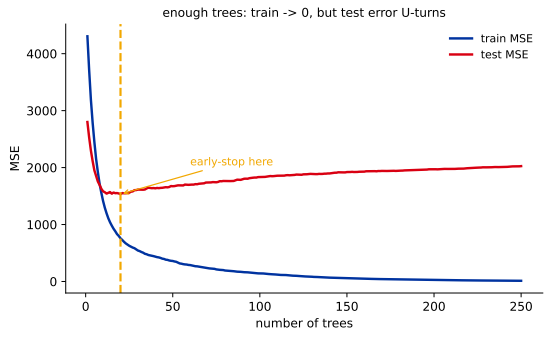
In [18], *more trees never hurt* a random forest. Same for boosting?

Predict-first: does boosting overfit with more trees?

In [18], *more trees never hurt* a random forest. Same for boosting?

No – the opposite. Training error drives toward **0** (it can memorize); test error drops, then **U-turns** and rises.

Boosting keeps fitting the *previous* errors, eventually fitting noise. Controlled by a small η + **early stopping**.



A high-capacity config (depth-3 trees, 250 rounds) on a noisy toy – cranked up to *show* the failure mode.

Contrast with random forests: RF averages *independent* trees → more is safe. Boosting adds *dependent* trees that chase residuals → more can overfit.

The reframe: you were doing gradient descent

Why is fitting residuals a good idea? Look at the squared loss $L = \frac{1}{2}(y - F)^2$:

$$\frac{\partial L}{\partial F} = -(y - F) \quad \Longrightarrow \quad -\frac{\partial L}{\partial F} = y - F = \textbf{the residual}.$$

(The $\frac{1}{2}$ is the convention that makes this come out with no stray factor of 2.)

So the residual you have been fitting **is the negative gradient** of the loss. Each tree takes one step **downhill** – you were doing gradient descent all along, just in *function space* (adding functions) instead of parameter space.

Function-space gradient descent (Friedman, 2001)

For $m = 1, 2, \ldots, M$:

1. **Pseudo-residuals** (the negative gradient at the current model):

$$r_{im} = -\left[\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)}\right]_{F=F_{m-1}}$$

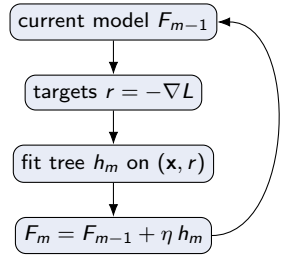
2. Fit a base learner h_m to the pseudo-residuals r_{im}.
3. **Line search** for the best step: $\gamma_m = \arg\min_\gamma \sum_{i=1}^{n} L(y_i, F_{m-1}(\mathbf{x}_i) + \gamma\, h_m(\mathbf{x}_i))$.
4. Update: $F_m = F_{m-1} + \eta\, \gamma_m\, h_m$.

What sklearn actually does (a refinement): it fits the tree, then sets *each leaf* to its own optimal step (a per-leaf line search), and shrinks all leaves by η. For squared loss that per-leaf step is just the leaf's **residual mean** – exactly the by-hand round you did two slides ago.

So what do we actually fit each round?

Cut through the notation – every round is the *same* three moves:

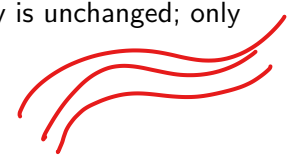
1. **target** for each row = the negative gradient at the current model (the "pseudo-residual" r_{im});
2. fit a **plain regression tree** to $(\mathbf{x},\ r)$ – ordinary inputs, residual-like targets;
3. add it, shrunk by η.



"Fitting the residuals" **is** gradient descent – one and the same procedure. For **squared loss** the target is literally $y - F$ (the ordinary residual); for **log-loss** it is $y - p$. Only the *target formula* changes; the base learner is always a vanilla regression tree on $(\mathbf{x},$ target$)$.

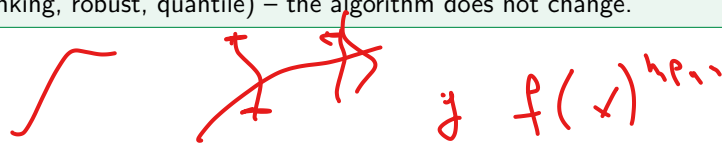
Any loss you like

Swap the loss ⇒ swap the pseudo-residual. The whole machinery is unchanged; only "what each tree chases" changes.



- **Squared loss** → pseudo-residual $= y - F$ (the plain residual).
- **Absolute / Huber loss** → pseudo-residual = the *sign* or a *clipped* residual → **robust to outliers** (one bad point can't drag the fit far).

This is boosting's superpower over AdaBoost: pick the loss that matches your problem (regression, classification, ranking, robust, quantile) – the algorithm does not change.

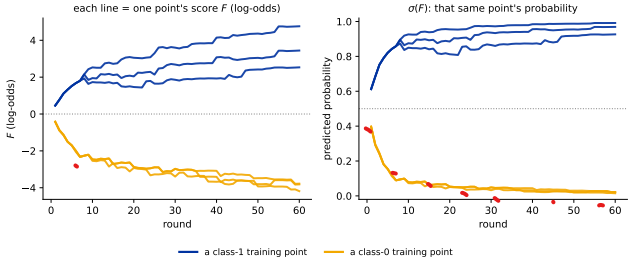


Gradient boosting for classification

Same machinery, **log-loss** instead of squared loss:

- each tree still fits the pseudo-residual, now $\mathbf{y} - \mathbf{p}$ (with $p = \sigma(F)$);
- the trees sum into F, which lives in **log-odds** space (any real number);
- apply the **sigmoid once, at the very end** to read off a probability.

In the plot **each line follows one training point** (3 per class): F climbs for class-1 points and falls for class-0 points; $\sigma(F)$ then heads to 1 or 0.



CART

Regression chases $y - F$ and predicts F directly; classification chases $y - p$, accumulates F in log-odds, and squashes with σ at the end. **Same gradient descent** – this is what the Titanic homework runs.

Regularization

More power than you want – three knobs hold it back.

Three knobs

knob	what it controls
`n_estimators` (M)	number of trees / rounds. More = lower bias, but eventually overfits ⇒ use **early stopping**.
`max_depth`	per-tree depth = **interaction order** (depth 1 = main effects only; deeper = feature interactions). Boosting likes *shallow*.
`learning_rate` (η)	**shrinkage**: how big a step each tree takes. Smaller = steadier, generalizes better.

The big one is the pair (η, M): they trade off – **smaller η needs more trees**, but usually generalizes better. Standard recipe: **small η + many trees + early stopping**.

Stochastic gradient boosting

Add a dash of **bagging**: each round, fit the tree on a random **subsample** of the rows (and optionally columns), not all of them.

- Faster (smaller fits) and a **regularizer** (the randomness decorrelates the trees, callback [18]).
- A bagging + boosting hybrid (Friedman, 2002).

Heads-up: this is NOT on by default – XGBoost's `subsample` and LightGBM's `bagging_fraction` both default to 1.0 (no subsampling). The libraries make it easy; **how they use it is [20]**.

Practice and the bridge ahead

Just enough to run it – the libraries get their own lecture.

In practice: use the libraries, not sklearn

You now understand the algorithm – but in real tabular work you will almost **never** hand-roll it or reach for sklearn's basic `GradientBoosting`.

Everyone uses the production libraries – **XGBoost**, **LightGBM**, **CatBoost** – the *same* gradient boosting, made fast and regularized. So we skip the sklearn API and go straight to them.

The one idea you always need – **early stopping** to pick the number of trees (callback: cross-validation) – is built into all of them. Their engineering, tuning, and a bake-off are the **whole next lecture ([20])**.

Bridge: the deep library session ([20])

Everything so far is the **algorithm**. Next lecture is the **engineering**:

- **XGBoost / LightGBM / CatBoost** – the libraries that win on tabular data: speed (histogram + leaf-wise growth), regularization baked into the objective, native categorical handling.
- How to **tune** them (the knob order that matters) and a real bake-off.
- **Stacking** – combining different model types – to close the chapter.

Next ([20]): same gradient boosting you just learned – made fast, regularized, and production-ready.

Recap

- **Boosting** adds shallow trees **in sequence**, each fitting the previous ensemble's leftover error; it cuts **bias**.
- That "leftover error" is the **negative gradient** of the loss – boosting is gradient descent in function space (any loss: $y - F$ regression, $y - p$ classification).
- More trees **can overfit** (unlike RF) – control with small η, shallow trees, and **early stopping**.

Workflow: small η + many trees + early stopping; keep trees shallow; add row/column subsampling if it helps. For the production versions, see **[20]**.